

23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

3. Database Design

Group No: C5

Project Title: Smart Campus Resource Booking System

Team Members:

Roll No	Student Name
AM.SC.U4CSE25201	AKSHANYA S
AM.SC.U4CSE25229	KRISHNA JAYAKUMAR
AM.SC.U4CSE25215	DHARUN A

3.1 Detailed Description of the Project

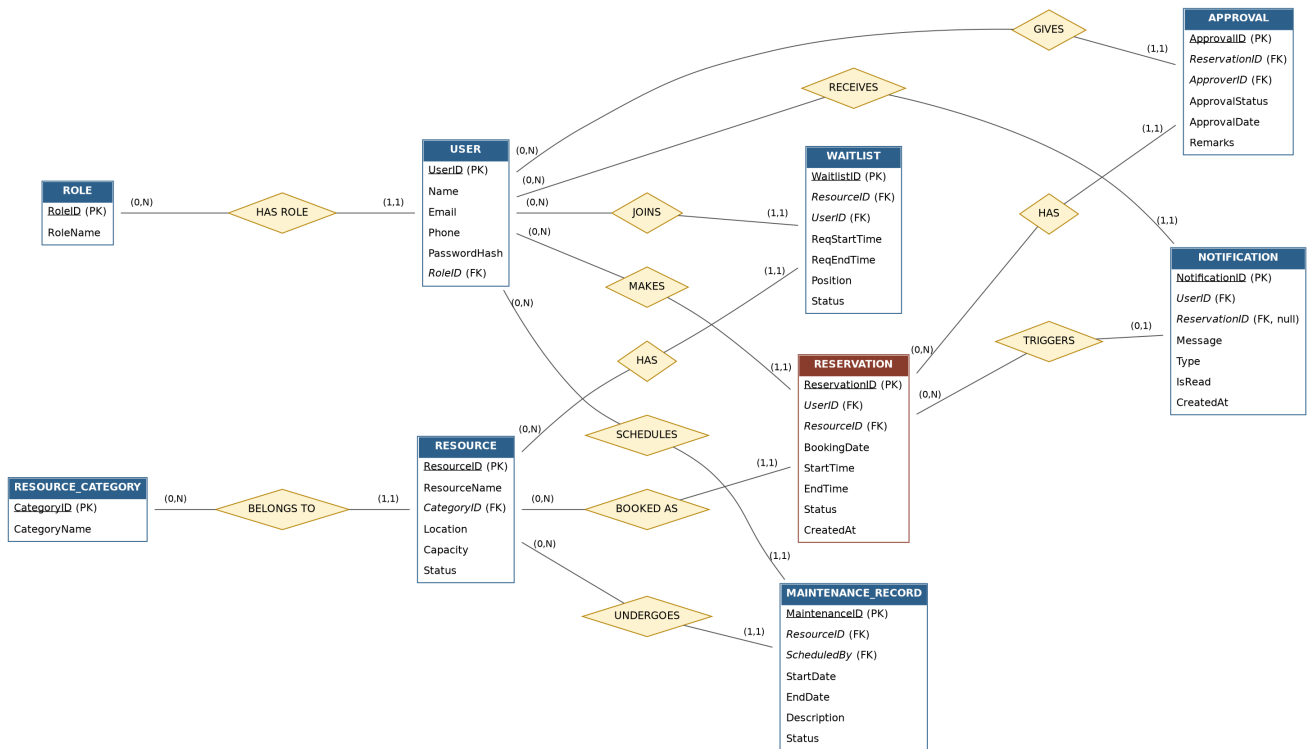
Campus resources like classrooms, labs, study rooms, sports venues, equipment and parking are currently tracked through manual registers, causing double bookings and no real-time visibility. The Smart Campus Resource Booking System replaces this with a single database-backed platform used by two kinds of people: Students/Faculty, who search for and reserve resources, and Admin/Maintenance Staff, who approve requests, schedule maintenance and view utilisation reports. Every person is stored as a User (Name, Email, Phone, Password) and linked to exactly one Role, which decides what they can do.

Resources (Name, Location, Capacity, Status) are grouped under Resource Categories, and each Reservation links a User to a Resource for a given BookingDate/StartTime/EndTime with a Status (Pending /Approved /Rejected / Cancelled /Completed).

Business rules: the system never allows two reservations on the same resource to overlap while both are pending or approved. Shared resources go through an Approval step recorded against the reservation and the approver — since a rejected reservation can be resubmitted, one reservation can end up with more than one approval record. If a slot is already taken, a user can join a Waitlist and is notified automatically when it frees up, and a resource under an active Maintenance Record cannot be booked for that period. Notifications keep everyone informed of confirmations, rejections, approvals and waitlist updates.

3.2 ER Diagram

PK is underlined, FK is italicised. The pair next to each entity gives cardinality and participation — (1,1) = total, (0,N)/ (0,1) = partial.



3.3 Relational Schema

ROLE (RoleID, RoleName)

USER (UserID, Name, Email, Phone, Password, RoleID*)

RESOURCE_CATEGORY (CategoryID, CategoryName)

RESOURCE (ResourceID, ResourceName, CategoryID*, Location, Capacity, Status)

RESERVATION (ReservationID, UserID*, ResourceID*, BookingDate, StartTime, EndTime, Status, CreatedAt)

APPROVAL (ApprovalID, ReservationID*, ApproverID*, ApprovalStatus, ApprovalDate, Remarks)

WAITLIST (WaitlistID, ResourceID*, UserID*, ReqStartTime, ReqEndTime, Position, Status)

MAINTENANCE_RECORD (MaintenanceID, ResourceID*, ScheduledBy*, StartDate, EndDate, Description, Status)

NOTIFICATION (NotificationID, UserID*, ReservationID*, Message, Type, IsRead, CreatedAt)

(* = foreign key; primary key of each relation is underlined below.)

Column	Data Type	Constraint
ROLE		
RoleID	SERIAL	PRIMARY KEY
RoleName	VARCHAR(50)	NOT NULL, UNIQUE
USER		
UserID	SERIAL	PRIMARY KEY
Name	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	NOT NULL, UNIQUE

Column	Data Type	Constraint
Phone	VARCHAR(15)	—
Password	VARCHAR(255)	NOT NULL
<i>RoleID</i>	INT	NOT NULL, FK → ROLE(RoleID)
RESOURCE_CATEGORY		
CategoryID	SERIAL	PRIMARY KEY
CategoryName	VARCHAR(50)	NOT NULL, UNIQUE
RESOURCE		
ResourceID	SERIAL	PRIMARY KEY
ResourceName	VARCHAR(100)	NOT NULL
<i>CategoryID</i>	INT	NOT NULL, FK → RESOURCE_CATEGORY
Location	VARCHAR(100)	—
Capacity	INT	CHECK (Capacity > 0)
Status	VARCHAR(20)	CHECK IN ('Available','Under Maintenance','Inactive'), DEFAULT 'Available'
RESERVATION		
ReservationID	SERIAL	PRIMARY KEY
<i>UserID</i>	INT	NOT NULL, FK → USER
<i>ResourceID</i>	INT	NOT NULL, FK → RESOURCE
BookingDate	DATE	NOT NULL
StartTime	TIMESTAMP	NOT NULL
EndTime	TIMESTAMP	NOT NULL, CHECK (EndTime > StartTime)
Status	VARCHAR(20)	CHECK IN ('Pending','Approved','Rejected','Cancelled','Completed')
CreatedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
APPROVAL		
ApprovalID	SERIAL	PRIMARY KEY
<i>ReservationID</i>	INT	NOT NULL, FK → RESERVATION
<i>ApproverID</i>	INT	NOT NULL, FK → USER
ApprovalStatus	VARCHAR(20)	CHECK IN ('Approved','Rejected','Pending')
ApprovalDate	TIMESTAMP	—
Remarks	VARCHAR(255)	—
WAITLIST		
WaitlistID	SERIAL	PRIMARY KEY
<i>ResourceID</i>	INT	NOT NULL, FK → RESOURCE
<i>UserID</i>	INT	NOT NULL, FK → USER
ReqStartTime	TIMESTAMP	NOT NULL
ReqEndTime	TIMESTAMP	NOT NULL, CHECK (> ReqStartTime)
Position	INT	NOT NULL

Column	Data Type	Constraint
Status	VARCHAR(20)	CHECK IN ('Waiting','Notified','Expired','Converted')
MAINTENANCE_RECORD		
MaintenanceID	SERIAL	PRIMARY KEY
<i>ResourceID</i>	INT	NOT NULL, FK → RESOURCE
<i>ScheduledBy</i>	INT	NOT NULL, FK → USER
StartDate	DATE	NOT NULL
EndDate	DATE	NOT NULL, CHECK (>= StartDate)
Description	VARCHAR(255)	—
Status	VARCHAR(20)	CHECK IN ('Scheduled','In Progress','Completed','Cancelled')
NOTIFICATION		
NotificationID	SERIAL	PRIMARY KEY
<i>UserID</i>	INT	NOT NULL, FK → USER
<i>ReservationID</i>	INT	NULL, FK → RESERVATION
Message	VARCHAR(255)	NOT NULL
Type	VARCHAR(30)	—
IsRead	BOOLEAN	DEFAULT FALSE
CreatedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

- No two rows in RESERVATION for the same ResourceID may have overlapping time ranges while both are Pending/Approved (no double booking).
- UNIQUE (ReservationID, ApproverID) in APPROVAL — the same approver reviews a given reservation only once.

3.4 Normalization and Functional Dependency Analysis

Since one reservation can be reviewed by more than one approver over time, this register has a repeating group of approval details per reservation.

UNF

```
BOOKING_REGISTER (ReservationID, UserID, UserName, UserEmail, RoleID, RoleName,
ResourceID,
ResourceName, CategoryID, CategoryName, Location, Capacity, BookingDate, StartTime,
EndTime,
Status, { ApproverID, ApproverName, ApprovalStatus, ApprovalDate } )
```

1NF

Removing the repeating approval group (one row per ReservationID-ApproverID pair) gives an all-atomic relation:

```
BOOKING_REGISTER_1NF (ReservationID, ApproverID, UserID, UserName, UserEmail, RoleID,
RoleName,
ResourceID, ResourceName, CategoryID, CategoryName, Location, Capacity, BookingDate,
StartTime, EndTime, Status, ApproverName, ApprovalStatus, ApprovalDate)
```

Functional dependencies:

- F1: ReservationID → UserID, ResourceID, BookingDate, StartTime, EndTime, Status
- F2: UserID → UserName, UserEmail, RoleID
- F3: RoleID → RoleName
- F4: ResourceID → ResourceName, CategoryID, Location, Capacity
- F5: CategoryID → CategoryName
- F6: ApproverID → ApproverName
- F7: ReservationID, ApproverID → ApprovalStatus, ApprovalDate

ReservationID alone can't be the key: it never determines ApprovalStatus/ApprovalDate, since one reservation can have several approvers. Taking the closure of {ReservationID, ApproverID} through F1–F7 covers every attribute, and no smaller set does, so the primary key is {ReservationID, ApproverID}.

2NF

UserID, ResourceID, BookingDate, StartTime, EndTime, Status (and everything reachable from them) depend on ReservationID alone; ApproverName depends on ApproverID alone. Both are partial dependencies on the composite key, so this violates 2NF. Splitting on the true determinant:

RESERVATION_2NF (ReservationID, UserID, ResourceID, BookingDate, StartTime, EndTime, Status, ...UserName, ResourceName, etc.)

APPROVER_2NF (ApproverID, ApproverName)

APPROVAL_2NF (ReservationID, ApproverID, ApprovalStatus, ApprovalDate)

3NF

RESERVATION_2NF still has transitive dependencies: ReservationID → UserID → {UserName, UserEmail, RoleID} → RoleName, and ReservationID → ResourceID → {ResourceName, CategoryID, Location, Capacity} → CategoryName. Removing these — and dropping APPROVER_2NF, since an approver is just a User already in the USER table — gives the final relations, matching Section 3.3:

RESERVATION, USER, ROLE, RESOURCE, RESOURCE_CATEGORY, APPROVAL

BCNF

Checking that every determinant is a candidate key:

Relation	Determinant	Key? → BCNF?
RESERVATION	ReservationID → all other columns	ReservationID is the only key → BCNF ✓
USER	UserID → all; Email → all (Email is unique)	Both are candidate keys → BCNF ✓
ROLE	RoleID → RoleName	Only key → BCNF ✓
RESOURCE	ResourceID → all other columns	Only key → BCNF ✓
RESOURCE_CATEGORY	CategoryID → CategoryName	Only key → BCNF ✓
APPROVAL	(ReservationID, ApproverID) → rest	Whole key is the only determinant → BCNF ✓

All relations satisfy BCNF, so the final design in Section 3.3 is free of redundancy from non-key dependencies.